import pandas as pd

df1 = Transactions (The "Sales" data)

df1 = pd.read_excel('QVI_transaction_data.xlsx')

df2 = Customers (The "Behavior" data)

df2 = pd.read_csv('QVI_purchase_behaviour.csv')

In [5]:
```
df1.info()
df2.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 264836 entries, 0 to 264835
Data columns (total 8 columns):
 #   Column          Non-Null Count   Dtype  
---  ------          --------------   -----  
 0   DATE            264836 non-null  int64  
 1   STORE_NBR       264836 non-null  int64  
 2   LYLTY_CARD_NBR  264836 non-null  int64  
 3   TXN_ID          264836 non-null  int64  
 4   PROD_NBR        264836 non-null  int64  
 5   PROD_NAME       264836 non-null  str    
 6   PROD_QTY        264836 non-null  int64  
 7   TOT_SALES       264836 non-null  float64
dtypes: float64(1), int64(6), str(1)
memory usage: 16.2 MB
<class 'pandas.DataFrame'>
RangeIndex: 72637 entries, 0 to 72636
Data columns (total 3 columns):
 #   Column            Non-Null Count  Dtype 
---  ------            --------------  ----- 
 0   LYLTY_CARD_NBR    72637 non-null  int64 
 1   LIFESTAGE         72637 non-null  str   
 2   PREMIUM_CUSTOMER  72637 non-null  str   
dtypes: int64(1), str(2)
memory usage: 1.7 MB
```

In [3]:
```
df1.head()
```

Out[3]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PRO
0	43390	1	1000	1	5	Natural Chip Compny SeaSalt175g	
1	43599	1	1307	348	66	CCs Nacho Cheese 175g	
2	43605	1	1343	383	61	Smiths Crinkle Cut Chips Chicken 170g	
3	43329	2	2373	974	69	Smiths Chip Thinly S/Cream&Onion 175g	
4	43330	2	2426	1038	108	Kettle Tortilla ChpsHny&Jlpno Chili 150g	

In [4]:
```
df2.head()
```

Out[4]:

	LYLTY_CARD_NBR	LIFESTAGE	PREMIUM_CUSTOMER
0	1000	YOUNG SINGLES/COUPLES	Premium
1	1002	YOUNG SINGLES/COUPLES	Mainstream
2	1003	YOUNG FAMILIES	Budget
3	1004	OLDER SINGLES/COUPLES	Mainstream
4	1005	MIDAGE SINGLES/COUPLES	Mainstream

In [6]:
```
# Convert integer dates to actual datetime objects
df1['DATE'] = pd.to_datetime(df1['DATE'], unit='D', origin='1899-12-30')

# Run this to verify the first few dates look normal now
df1['DATE'].head()
```

Out[6]:
```
0   2018-10-17
1   2019-05-14
2   2019-05-20
3   2018-08-17
4   2018-08-18
Name: DATE, dtype: datetime64[s]
```

In [7]:
```
# Look at unique product names to see if 'salsa' is in there
# This prints every unique product name in the dataset
print(df1['PROD_NAME'].unique())
```

```
<StringArray>
[  'Natural Chip        Compny SeaSalt175g',
                'CCs Nacho Cheese    175g',
   'Smiths Crinkle Cut  Chips Chicken 170g',
   'Smiths Chip Thinly  S/Cream&Onion 175g',
 'Kettle Tortilla ChpsHny&Jlpno Chili 150g',
 'Old El Paso Salsa   Dip Tomato Mild 300g',
 'Smiths Crinkle Chips Salt & Vinegar 330g',
    'Grain Waves         Sweet Chilli 210g',
  'Doritos Corn Chip Mexican Jalapeno 150g',
    'Grain Waves Sour    Cream&Chives 210G',
 ...
         'Doritos Cheese      Supreme 330g',
      'Smiths Crinkle Cut  Snag&Sauce 150g',
   'WW Sour Cream &OnionStacked Chips 160g',
                'RRD Lime & Pepper   165g',
   'Natural ChipCo Sea  Salt & Vinegr 175g',
    'Red Rock Deli Chikn&Garlic Aioli 150g',
      'RRD SR Slow Rst     Pork Belly 150g',
                'RRD Pc Sea Salt     165g',
       'Smith Crinkle Cut   Bolognese 150g',
                'Doritos Salsa Mild  300g']
Length: 114, dtype: str
```

In [8]:
```
# This creates a temporary view of only the salsa products
salsa_products = df1[df1['PROD_NAME'].str.contains("salsa", case=False)]
print(f"Number of salsa transactions found: {len(salsa_products)}")
print(salsa_products['PROD_NAME'].unique())
```

```
Number of salsa transactions found: 18094
<StringArray>
['Old El Paso Salsa   Dip Tomato Mild 300g',
 'Red Rock Deli SR    Salsa & Mzzrlla 150g',
    'Smiths Crinkle Cut  Tomato Salsa 150g',
          'Doritos Salsa       Medium 300g',
 'Old El Paso Salsa   Dip Chnky Tom Ht300g',
           'Woolworths Mild     Salsa 300g',
  'Old El Paso Salsa   Dip Tomato Med 300g',
           'Woolworths Medium   Salsa 300g',
                'Doritos Salsa Mild  300g']
Length: 9, dtype: str
```

In [9]:
```
# The ~ symbol means "NOT"
df1 = df1[~df1['PROD_NAME'].str.lower().str.contains("salsa")]
```

In [10]:
```
print(df1['PROD_NAME'].unique())
```

```
<StringArray>
[  'Natural Chip        Compny SeaSalt175g',
                'CCs Nacho Cheese    175g',
   'Smiths Crinkle Cut  Chips Chicken 170g',
   'Smiths Chip Thinly  S/Cream&Onion 175g',
 'Kettle Tortilla ChpsHny&Jlpno Chili 150g',
 'Smiths Crinkle Chips Salt & Vinegar 330g',
    'Grain Waves         Sweet Chilli 210g',
  'Doritos Corn Chip Mexican Jalapeno 150g',
    'Grain Waves Sour    Cream&Chives 210G',
    'Kettle Sensations   Siracha Lime 150g',
 ...
                'RRD Salt & Vinegar  165g',
         'Doritos Cheese      Supreme 330g',
      'Smiths Crinkle Cut  Snag&Sauce 150g',
   'WW Sour Cream &OnionStacked Chips 160g',
                'RRD Lime & Pepper   165g',
   'Natural ChipCo Sea  Salt & Vinegr 175g',
    'Red Rock Deli Chikn&Garlic Aioli 150g',
      'RRD SR Slow Rst     Pork Belly 150g',
                'RRD Pc Sea Salt     165g',
       'Smith Crinkle Cut   Bolognese 150g']
Length: 105, dtype: str
```

In [11]:
```
df1.describe()
```

Out[11]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_C
count	246742	246742.000000	2.467420e+05	2.467420e+05	246742.000000	246742.000
mean	2018-12-30 01:19:01	135.051098	1.355310e+05	1.351311e+05	56.351789	1.908
min	2018-07-01 00:00:00	1.000000	1.000000e+03	1.000000e+00	1.000000	1.000
25%	2018-09-30 00:00:00	70.000000	7.001500e+04	6.756925e+04	26.000000	2.000
50%	2018-12-30 00:00:00	130.000000	1.303670e+05	1.351830e+05	53.000000	2.000
75%	2019-03-31 00:00:00	203.000000	2.030840e+05	2.026538e+05	87.000000	2.000
max	2019-06-30 00:00:00	272.000000	2.373711e+06	2.415841e+06	114.000000	200.000
std	NaN	76.787096	8.071528e+04	7.814772e+04	33.695428	0.659

In [12]:
```
# Show rows where someone bought more than 10 packets at once
outliers = df1[df1['PROD_QTY'] > 10]
outliers
```

Out[12]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QT
69762	2018-08-19	226	226000	226201	4	Dorito Corn Chp Supreme 380g	2(
69763	2019-05-20	226	226000	226210	4	Dorito Corn Chp Supreme 380g	2(

In [13]:
```
# Filter out the wholesale customer
df1 = df1[df1['LYLTY_CARD_NBR'] != 226000]

# Double check that the max quantity is now reasonable
print("New max quantity:", df1['PROD_QTY'].max())
```

```
New max quantity: 5
```

In [15]:
```
# Extract digits from the PROD_NAME column
df1['PACK_SIZE'] = df1['PROD_NAME'].str.extract(r'(\d+)').astype(float)

# Check the unique pack sizes to make sure it worked
print(df1['PACK_SIZE'].unique())
```

```
[175. 170. 150. 330. 210. 270. 220. 125. 110. 134. 380. 180. 165. 135.
 250. 200. 160. 190.  90.  70.]
```

In [16]:
```
# Create a BRAND column by taking the first word of PROD_NAME
df1['BRAND'] = df1['PROD_NAME'].str.split().str[0]

# Look at the unique brands to find the "doubles"
print(df1['BRAND'].unique())
```

```
['Natural' 'CCs' 'Smiths' 'Kettle' 'Grain' 'Doritos' 'Twisties' 'WW'
 'Thins' 'Burger' 'NCC' 'Cheezels' 'Infzns' 'Red' 'Pringles' 'Dorito'
 'Infuzions' 'Smith' 'GrnWves' 'Tyrrells' 'Cobs' 'French' 'RRD' 'Tostitos'
 'Cheetos' 'Woolworths' 'Snbts' 'Sunbites']
```

In [17]:
```
# Create a dictionary to map the "messy" names to a single "clean" name
brand_map = {
    "Red": "RRD",
    "Dorito": "Doritos",
    "Infzns": "Infuzions",
    "Smith": "Smiths",
    "Snbts": "Sunbites",
    "WW": "Woolworths",
    "Natural": "NCC",
    "Grain": "GrnWves"
}
```

```
# Apply the mapping to your BRAND column
df1['BRAND'] = df1['BRAND'].replace(brand_map)

# Check the unique values again to ensure they are merged
print(df1['BRAND'].unique())
```

```
['NCC' 'CCs' 'Smiths' 'Kettle' 'GrnWves' 'Doritos' 'Twisties' 'Woolworths'
 'Thins' 'Burger' 'Cheezels' 'Infuzions' 'RRD' 'Pringles' 'Tyrrells'
 'Cobs' 'French' 'Tostitos' 'Cheetos' 'Sunbites']
```

In [18]:
```
# Check if any rows are missing information
print(df2.isnull().sum())
```

```
LYLTY_CARD_NBR      0
LIFESTAGE           0
PREMIUM_CUSTOMER    0
dtype: int64
```

In [19]:
```
# Check if loyalty cards are unique
print("Total rows:", len(df2))
print("Unique cards:", df2['LYLTY_CARD_NBR'].nunique())

# If those two numbers aren't the same, you have duplicates!
```

```
Total rows: 72637
Unique cards: 72637
```

In [20]:
```
# See the different types of customers
print(df2['LIFESTAGE'].unique())
print(df2['PREMIUM_CUSTOMER'].unique())
```

```
<StringArray>
[ 'YOUNG SINGLES/COUPLES',         'YOUNG FAMILIES',  'OLDER SINGLES/COUPLES',
 'MIDAGE SINGLES/COUPLES',           'NEW FAMILIES',         'OLDER FAMILIES',
               'RETIREES']
Length: 7, dtype: str
<StringArray>
['Premium', 'Mainstream', 'Budget']
Length: 3, dtype: str
```

In [21]:
```
# Merge transactions (df1) with customer behavior (df2)
# We use a 'left' join to keep all transaction records
data = pd.merge(df1, df2, on='LYLTY_CARD_NBR', how='left')

# Check for any missing values after the merge
print(data.isnull().sum())
```

```
DATE                0
STORE_NBR           0
LYLTY_CARD_NBR      0
TXN_ID              0
PROD_NBR            0
PROD_NAME           0
PROD_QTY            0
TOT_SALES           0
PACK_SIZE           0
BRAND               0
LIFESTAGE           0
PREMIUM_CUSTOMER    0
dtype: int64
```

In [22]:
```
# Summary of total sales, number of customers, and average price per unit
summary = data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER']).agg(
    Total_Sales=('TOT_SALES', 'sum'),
    Unique_Customers=('LYLTY_CARD_NBR', 'nunique'),
    Average_Price_Per_Unit=('TOT_SALES', lambda x: x.sum() / data.loc[x.index, 'PRO
).reset_index()

print(summary.sort_values(by='Total_Sales', ascending=False))
```

	LIFESTAGE	PREMIUM_CUSTOMER	Total_Sales	Unique_Customers	Average_Price_Per_Unit
6	OLDER FAMILIES	Budget	156863.75	4611	3.747969
19	YOUNG SINGLES/COUPLES	Mainstream	147582.20	7917	4.074043
13	RETIREES	Mainstream	145168.95	6358	3.852986
15	YOUNG FAMILIES	Budget	129717.95	3953	3.761903
9	OLDER SINGLES/COUPLES	Budget	127833.60	4849	3.887529
10	OLDER SINGLES/COUPLES	Mainstream	124648.50	4858	3.822753
11	OLDER SINGLES/COUPLES	Premium	123537.55	4682	3.897698
12	RETIREES	Budget	105916.30	4385	3.932731
7	OLDER FAMILIES	Mainstream	96413.55	2788	3.736380
14	RETIREES	Premium	91296.65	3812	3.924037
16	YOUNG FAMILIES	Mainstream	86338.25	2685	3.722439
1	MIDAGE SINGLES/COUPLES	Mainstream	84734.25	3298	3.994449
17	YOUNG FAMILIES	Premium	78571.70	2398	3.759232
8	OLDER FAMILIES	Premium	75242.60	2231	3.717703
18	YOUNG SINGLES/COUPLES	Budget	57122.10	3647	3.685297
2	MIDAGE SINGLES/COUPLES	Premium	54443.85	2369	3.780823
20	YOUNG SINGLES/COUPLES	Premium	39052.30	2480	3.692889
0	MIDAGE SINGLES/COUPLES	Budget	33345.70	1474	3.753878
3	NEW FAMILIES	Budget	20607.45	1087	3.931969
4	NEW FAMILIES	Mainstream	15979.70	830	3.935887
5	NEW FAMILIES	Premium	10760.80	575	3.886168

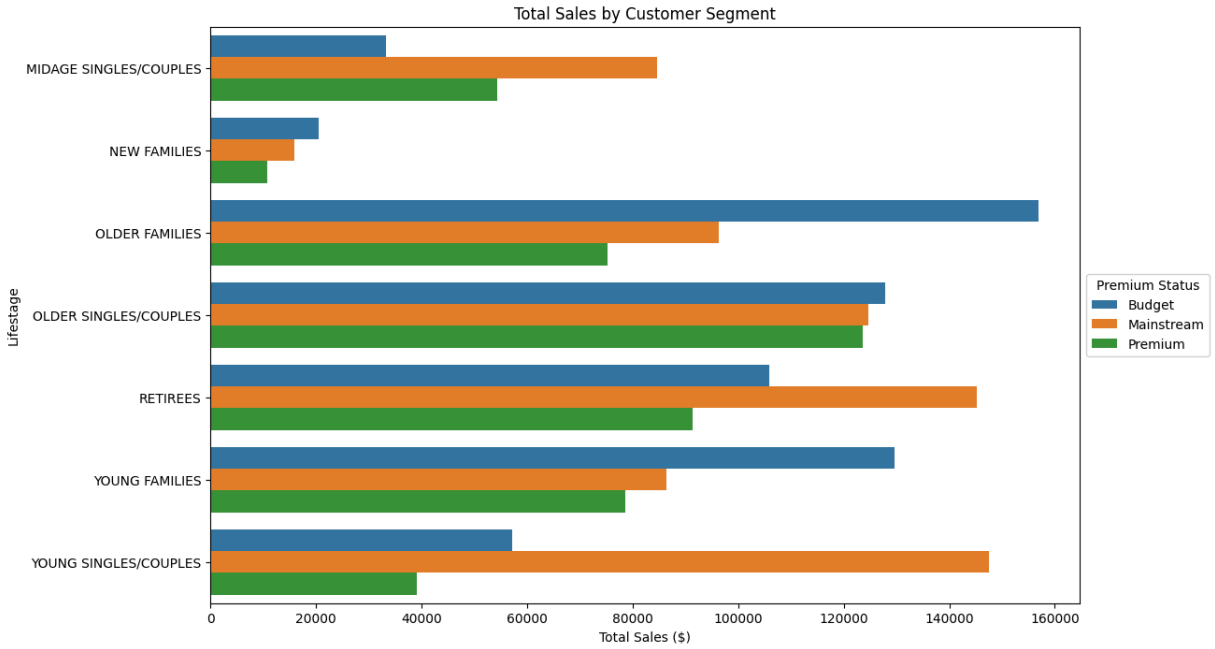
In [23]:
```
# Filter for the target segment
target_segment = data[(data['LIFESTAGE'] == 'YOUNG SINGLES/COUPLES') &
                      (data['PREMIUM_CUSTOMER'] == 'Mainstream')]

# See their top 5 brands
top_brands = target_segment['BRAND'].value_counts(normalize=True).head(5)
print("Top Brands for Mainstream Young Singles/Couples:")
print(top_brands)
```

```
Top Brands for Mainstream Young Singles/Couples:
BRAND
Kettle       0.196684
Doritos      0.121725
Pringles     0.118451
Smiths       0.098291
Infuzions    0.063958
Name: proportion, dtype: float64
```

In [24]:
```python
# Check pack size preferences for Mainstream Young Singles/Couples
pack_size_pref = target_segment['PACK_SIZE'].value_counts(normalize=True).head(5)

print("Top Pack Sizes for Mainstream Young Singles/Couples:")
print(pack_size_pref)
```

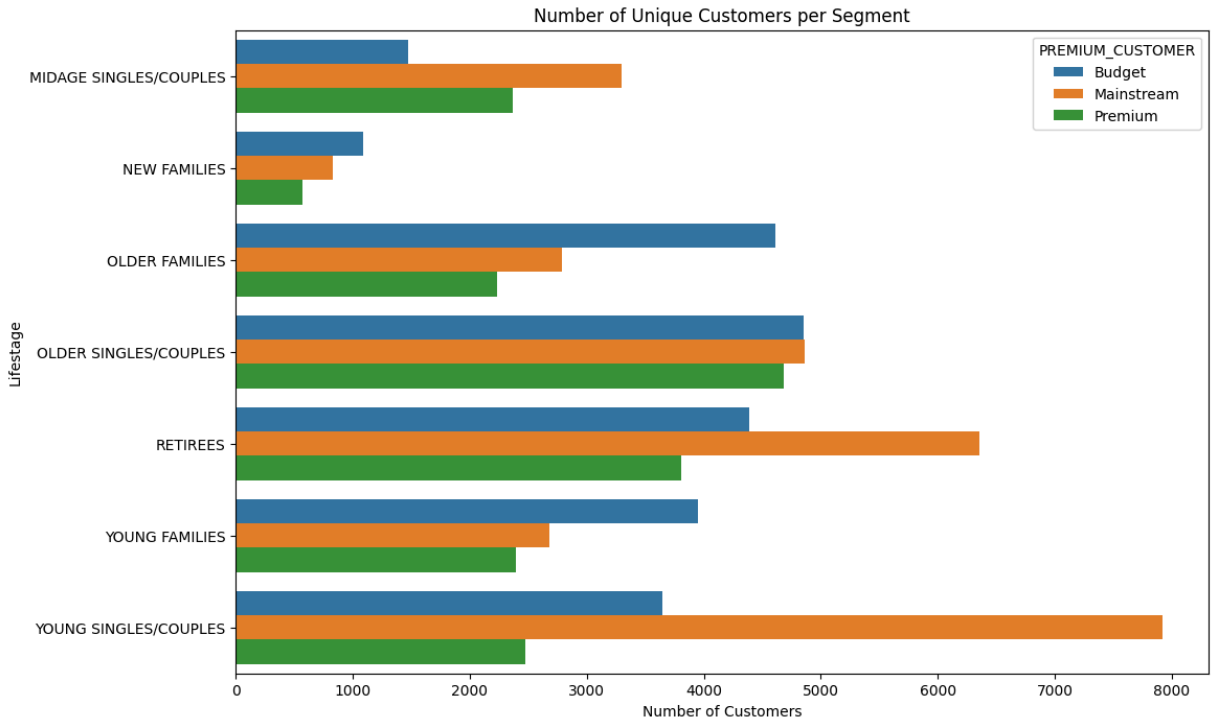
```
Top Pack Sizes for Mainstream Young Singles/Couples:
PACK_SIZE
175.0    0.255679
150.0    0.157593
134.0    0.118451
110.0    0.104943
170.0    0.080587
Name: proportion, dtype: float64
```

In [27]:
```python
import matplotlib.pyplot as plt
import seaborn as sns

# Create a grouped dataframe for plotting
plot_data = data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])['TOT_SALES'].sum().rese

# Set plot size and style
plt.figure(figsize=(12, 8))
sns.barplot(data=plot_data, y='LIFESTAGE', x='TOT_SALES', hue='PREMIUM_CUSTOMER')

# Add labels and title
plt.title('Total Sales by Customer Segment')
plt.xlabel('Total Sales ($)')
plt.ylabel('Lifestage')
plt.legend(title='Premium Status', loc='center left', bbox_to_anchor=(1, 0.5))
plt.show()
```



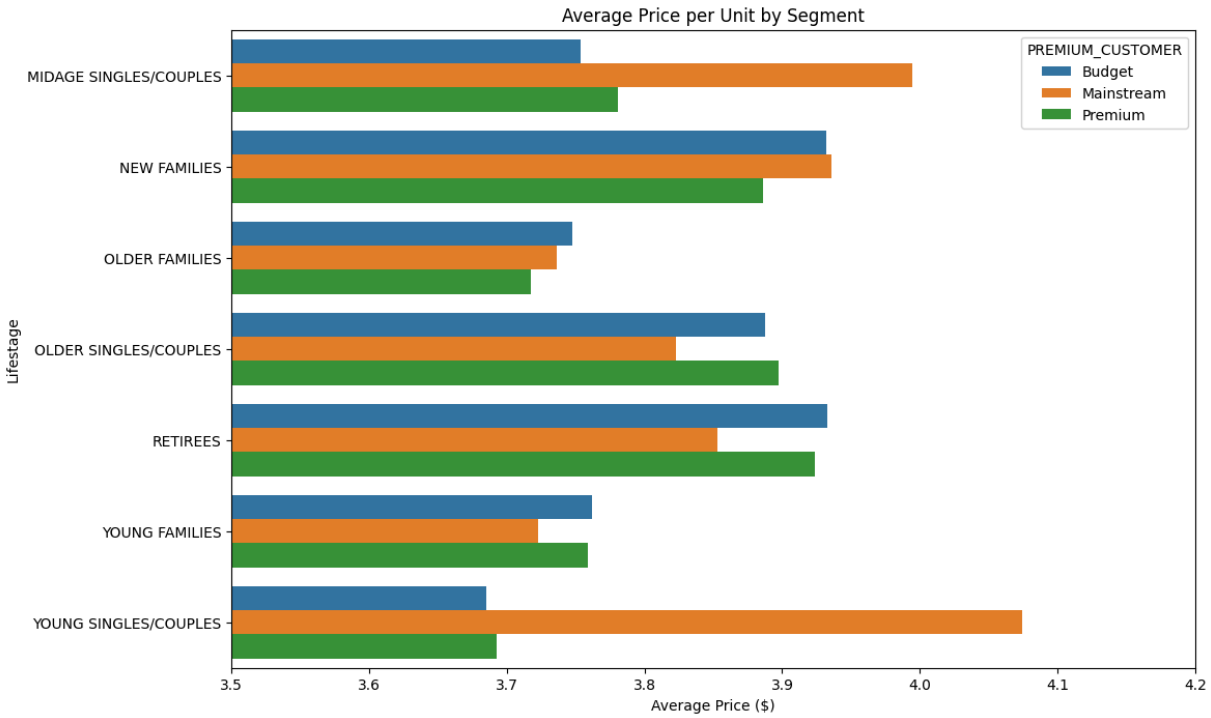
In [ ]:
```
# Total Sales by Segment (Bar Chart)
#This shows exactly where the money is coming from across those 21 segments.
```

In [28]:
```
# Group by segments to count unique loyalty cards
customer_counts = data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])['LYLTY_CARD_NBR']

plt.figure(figsize=(12, 8))
sns.barplot(data=customer_counts, y='LIFESTAGE', x='LYLTY_CARD_NBR', hue='PREMIUM_C

plt.title('Number of Unique Customers per Segment')
plt.xlabel('Number of Customers')
plt.ylabel('Lifestage')
plt.show()
```



In [ ]:
```
# Number of Customers (To show the "Growth Opportunity")
# We found that Mainstream Young Singles have the most customers but aren't #1 in s
```

In [29]:
```
# Calculate average price: Total Sales / Total Quantity
avg_price = data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER']).apply(lambda x: x['TOT_
avg_price.columns = ['LIFESTAGE', 'PREMIUM_CUSTOMER', 'AVG_PRICE']

plt.figure(figsize=(12, 8))
sns.barplot(data=avg_price, y='LIFESTAGE', x='AVG_PRICE', hue='PREMIUM_CUSTOMER')

plt.title('Average Price per Unit by Segment')
plt.xlabel('Average Price ($)')
plt.ylabel('Lifestage')
plt.xlim(3.5, 4.2) # Zooming in to show the difference clearly
plt.show()
```



In [ ]:
```
#Average Price per Unit (The "Mainstream" Premium)
#This chart will visually prove that Mainstream Young Singles/Couples pay more for
```

In [32]:
```
from scipy.stats import ttest_ind

# Group 1: Mainstream Young Singles/Couples
mainstream_young = data[(data['LIFESTAGE'] == 'YOUNG SINGLES/COUPLES') & (data['PRE

# Group 2: Everyone else
others = data.loc[~((data['LIFESTAGE'] == 'YOUNG SINGLES/COUPLES') & (data['PREMIUM

# Perform t-test
t_stat, p_val = ttest_ind(mainstream_young, others.dropna())
print(f"P-value: {p_val}")
# If p-value < 0.05, the difference is statistically significant!
```

```
P-value: 1.419308440276165e-218
```

Executive Summary & Strategic Recommendations

Based on the data analysis of chip purchasing behavior across all customer segments, the following strategic recommendations are proposed for the upcoming category review:

1. Primary Target Segment: Mainstream Young Singles/Couples

- **The Insight:** While "Older Families - Budget" bring in the highest total revenue, **Mainstream Young Singles/Couples** represent the largest growth opportunity. They have the highest number of unique customers and the highest **Average Price per Unit ($4.07)**.

- **Statistical Significance:** A t-test confirmed a **p-value of 1.42e-218**, proving that this segment's preference for higher-priced chips is statistically significant and not due to random chance.

2. Product Optimization: Premium Brands & Specific Pack Sizes

- **The Insight:** This segment shows a clear preference for "Premium" flavored chips over traditional budget options. Their top brands are **Kettle** (19.7%), **Doritos**, and **Pringles**.
- **Pack Size Preference:** Sales are dominated by the **175g** size, followed by **134g** (the standard Pringles tube size).
- **Recommendation:** Focus the product mix on these specific brands and sizes to cater to the "Mainstream" taste profile.

3. Commercial Application & Placement

- **Targeting:** Implement marketing activations specifically for Mainstream Young Singles/Couples.
- **Placement:** Increase stock levels of Kettle and Pringles in stores located in high-density urban areas, near **major transit hubs**, and **commercial work centers** where young professionals shop.
- **Pricing Strategy:** Since this group is less price-sensitive, Julia can prioritize premium brand displays over deep-discount "Budget" brand promotions in these specific locations.

Task 2: Experimentation and Uplift Testing.

In [3]:
```
import pandas as pd
df = pd.read_csv('QVI_data.csv')
df.head()
```

Out[3]:

	LYLTY_CARD_NBR	DATE	STORE_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QTY
	1000	2018-10-17	1	1	5	Chip Natural Compny SeaSalt175g	2
	1002	2018-09-16	1	2	58	Red Rock Deli Chikn&Garlic Aioli 150g	1
	1003	2019-03-07	1	3	52	Sour Grain Waves Cream&Chives 210G	1
	1003	2019-03-08	1	4	106	ChipCo Natural Hony Soy Chckn175g	1
	1004	2018-11-02	1	5	96	WW Original Stacked Chips 160g	1

In [4]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 264834 entries, 0 to 264833
Data columns (total 12 columns):
 #   Column            Non-Null Count   Dtype  
---  ------            --------------   -----  
 0   LYLTY_CARD_NBR    264834 non-null  int64  
 1   DATE              264834 non-null  str    
 2   STORE_NBR         264834 non-null  int64  
 3   TXN_ID            264834 non-null  int64  
 4   PROD_NBR          264834 non-null  int64  
 5   PROD_NAME         264834 non-null  str    
 6   PROD_QTY          264834 non-null  int64  
 7   TOT_SALES         264834 non-null  float64
 8   PACK_SIZE         264834 non-null  int64  
 9   BRAND             264834 non-null  str    
 10  LIFESTAGE         264834 non-null  str    
 11  PREMIUM_CUSTOMER  264834 non-null  str    
dtypes: float64(1), int64(6), str(5)
memory usage: 24.2 MB
```

In [18]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load data
data = pd.read_csv('QVI_data.csv')

# Create YEARMONTH column (Format: YYYYMM)
data['YEARMONTH'] = pd.to_datetime(data['DATE']).dt.strftime('%Y%m').astype(int)

# Calculate measures over time
```

```
measure_over_time = data.groupby(['STORE_NBR', 'YEARMONTH']).agg(
    totSales=('TOT_SALES', 'sum'),
    nCustomers=('LYLTY_CARD_NBR', 'nunique'),
    nTxnPerCust=('LYLTY_CARD_NBR', 'count'),
    nChipsPerTxn=('PROD_QTY', 'sum'),
    avgPricePerUnit=('TOT_SALES', 'sum')
).reset_index()

# Finalize metric calculations
measure_over_time['nTxnPerCust'] = measure_over_time['nTxnPerCust'] / measure_over_
measure_over_time['nChipsPerTxn'] = measure_over_time['nChipsPerTxn'] / (measure_ov

# Filter to stores with full observation periods (12 months)
store_counts = measure_over_time.groupby('STORE_NBR').size()
stores_with_full_obs = store_counts[store_counts == 12].index
measure_over_time = measure_over_time[measure_over_time['STORE_NBR'].isin(stores_wi

# Define Pre-Trial Period
pre_trial_measures = measure_over_time[measure_over_time['YEARMONTH'] < 201902]
```

In [26]:

```
def calculate_correlation(input_table, metric_col, store_comparison):
    """Calculates correlation between a trial store and all other stores."""
    calc_corr_table = []
    trial_data = input_table[input_table['STORE_NBR'] == store_comparison][metric_c

    store_numbers = input_table['STORE_NBR'].unique()

    for i in store_numbers:
        control_data = input_table[input_table['STORE_NBR'] == i][metric_col].value
        # correlation requires at least 2 points and matching lengths
        corr = np.corrcoef(trial_data, control_data)[0, 1]
        calc_corr_table.append({'Store1': store_comparison, 'Store2': i, 'corr_meas

    return pd.DataFrame(calc_corr_table)

def calculate_magnitude_distance(input_table, metric_col, store_comparison):
    """Calculates standardized magnitude distance (1 - normalized distance)."""
    trial_data = input_table[input_table['STORE_NBR'] == store_comparison].sort_val
    store_numbers = input_table['STORE_NBR'].unique()
    dist_table = []

    for i in store_numbers:
        control_data = input_table[input_table['STORE_NBR'] == i].sort_values('YEAR

        # Calculate absolute difference
        diff = np.abs(trial_data[metric_col].values - control_data[metric_col].valu

        for idx, month in enumerate(trial_data['YEARMONTH']):
            dist_table.append({
                'Store1': store_comparison,
                'Store2': i,
                'YEARMONTH': month,
                'measure': diff[idx]
            })

    df_dist = pd.DataFrame(dist_table)
```

```
    # Standardize: 1 - (dist - min) / (max - min)
    df_dist['max_dist'] = df_dist.groupby(['Store1', 'YEARMONTH'])['measure'].trans
    df_dist['min_dist'] = df_dist.groupby(['Store1', 'YEARMONTH'])['measure'].trans
    df_dist['mag_measure'] = 1 - (df_dist['measure'] - df_dist['min_dist']) / (df_d

    return df_dist.groupby(['Store1', 'Store2'])['mag_measure'].mean().reset_index(
```

In [27]:
```
trial_store = 77

# Sales Metrics
corr_nSales = calculate_correlation(pre_trial_measures, 'totSales', trial_store)
mag_nSales = calculate_magnitude_distance(pre_trial_measures, 'totSales', trial_sto

# Customer Metrics
corr_nCust = calculate_correlation(pre_trial_measures, 'nCustomers', trial_store)
mag_nCust = calculate_magnitude_distance(pre_trial_measures, 'nCustomers', trial_st

# Create Score Table
score_nSales = pd.merge(corr_nSales, mag_nSales, on=['Store1', 'Store2'])
score_nSales['scoreNSales'] = (score_nSales['corr_measure'] * 0.5) + (score_nSales[

score_nCust = pd.merge(corr_nCust, mag_nCust, on=['Store1', 'Store2'])
score_nCust['scoreNCust'] = (score_nCust['corr_measure'] * 0.5) + (score_nCust['mag

# Final Combined Score
score_Control = pd.merge(score_nSales[['Store2', 'scoreNSales']],
                         score_nCust[['Store2', 'scoreNCust']], on='Store2')
score_Control['finalScore'] = (score_Control['scoreNSales'] * 0.5) + (score_Control

# Select Control Store (Highest score that isn't the trial store itself)
control_store_77 = score_Control[score_Control['Store2'] != trial_store].sort_value
print(f"Control Store for 77 is: {control_store_77}") # Should be 233
```

```
Control Store for 77 is: 233.0
```

In [28]:
```
# Scaling factor for Sales
trial_sum = pre_trial_measures[pre_trial_measures['STORE_NBR'] == trial_store]['tot
control_sum = pre_trial_measures[pre_trial_measures['STORE_NBR'] == control_store_7
scaling_factor = trial_sum / control_sum

# Apply scaling to control store
measure_sales = measure_over_time.copy()
scaled_control_sales = measure_sales[measure_sales['STORE_NBR'] == control_store_77
scaled_control_sales['controlSales'] = scaled_control_sales['totSales'] * scaling_f

# Calculate % Difference
trial_sales = measure_sales[measure_sales['STORE_NBR'] == trial_store][['YEARMONTH'
percentage_diff = pd.merge(scaled_control_sales[['YEARMONTH', 'controlSales']], tri
percentage_diff['percentageDiff'] = (percentage_diff['totSales'] - percentage_diff[

# Statistical Significance (t-test)
std_dev = percentage_diff[percentage_diff['YEARMONTH'] < 201902]['percentageDiff'].
percentage_diff['tValue'] = percentage_diff['percentageDiff'] / std_dev

# Trial period months (Feb, Mar, Apr)
```

```
trial_months = [201902, 201903, 201904]
print(percentage_diff[percentage_diff['YEARMONTH'].isin(trial_months)][['YEARMONTH'
```

```
   YEARMONTH    tValue
7     201902 -0.593520
8     201903  3.680430
9     201904  6.256669
```

In [30]:
```
trial_store = 86

# Sales Metrics
corr_nSales = calculate_correlation(pre_trial_measures, 'totSales', trial_store)
mag_nSales = calculate_magnitude_distance(pre_trial_measures, 'totSales', trial_sto

# Customer Metrics
corr_nCust = calculate_correlation(pre_trial_measures, 'nCustomers', trial_store)
mag_nCust = calculate_magnitude_distance(pre_trial_measures, 'nCustomers', trial_st

# Create Score Table
score_nSales = pd.merge(corr_nSales, mag_nSales, on=['Store1', 'Store2'])
score_nSales['scoreNSales'] = (score_nSales['corr_measure'] * 0.5) + (score_nSales[

score_nCust = pd.merge(corr_nCust, mag_nCust, on=['Store1', 'Store2'])
score_nCust['scoreNCust'] = (score_nCust['corr_measure'] * 0.5) + (score_nCust['mag

# Final Combined Score
score_Control = pd.merge(score_nSales[['Store2', 'scoreNSales']],
                         score_nCust[['Store2', 'scoreNCust']], on='Store2')
score_Control['finalScore'] = (score_Control['scoreNSales'] * 0.5) + (score_Control

# Select Control Store (Highest score that isn't the trial store itself)
control_store_86 = score_Control[score_Control['Store2'] != trial_store].sort_value
print(f"Control Store for 86 is: {control_store_86}")
```

```
Control Store for 86 is: 155.0
```

In [31]:
```
trial_store = 88

# Sales Metrics
corr_nSales = calculate_correlation(pre_trial_measures, 'totSales', trial_store)
mag_nSales = calculate_magnitude_distance(pre_trial_measures, 'totSales', trial_sto

# Customer Metrics
corr_nCust = calculate_correlation(pre_trial_measures, 'nCustomers', trial_store)
mag_nCust = calculate_magnitude_distance(pre_trial_measures, 'nCustomers', trial_st

# Create Score Table
score_nSales = pd.merge(corr_nSales, mag_nSales, on=['Store1', 'Store2'])
score_nSales['scoreNSales'] = (score_nSales['corr_measure'] * 0.5) + (score_nSales[

score_nCust = pd.merge(corr_nCust, mag_nCust, on=['Store1', 'Store2'])
score_nCust['scoreNCust'] = (score_nCust['corr_measure'] * 0.5) + (score_nCust['mag

# Final Combined Score
score_Control = pd.merge(score_nSales[['Store2', 'scoreNSales']],
                         score_nCust[['Store2', 'scoreNCust']], on='Store2')
score_Control['finalScore'] = (score_Control['scoreNSales'] * 0.5) + (score_Control
```

```
# Select Control Store (Highest score that isn't the trial store itself)
control_store_88 = score_Control[score_Control['Store2'] != trial_store].sort_value
print(f"Control Store for 88 is: {control_store_88}")
```

```
Control Store for 88 is: 237.0
```

In [34]:
```
def get_refined_score(trial_store, control_store, df):
    """Calculates the combined correlation and magnitude score."""
    # 1. Correlation
    trial_sales = df[df['STORE_NBR'] == trial_store]['totSales'].values
    control_sales = df[df['STORE_NBR'] == control_store]['totSales'].values
    corr_sales = np.corrcoef(trial_sales, control_sales)[0, 1]

    trial_customers = df[df['STORE_NBR'] == trial_store]['nCustomers'].values
    control_customers = df[df['STORE_NBR'] == control_store]['nCustomers'].values
    corr_customers = np.corrcoef(trial_customers, control_customers)[0, 1]

    # 2. Magnitude (Using your existing functions if preferred, or simplified here)
    # For speed in this loop, we'll focus on the average difference
    mag_sales = 1 - (np.abs(trial_sales - control_sales).sum() / trial_sales.sum())
    mag_customers = 1 - (np.abs(trial_customers - control_customers).sum() / trial_

    # 3. Combine into a final score
    final_score = (corr_sales + corr_customers + mag_sales + mag_customers) / 4
    return final_score

# Define the potential controls list (all stores except the trial ones)
trial_stores = [77, 86, 88]
potential_controls = [s for s in pre_trial_measures['STORE_NBR'].unique() if s not
```

In [36]:
```
def find_control_store(trial_store_nbr, potential_controls, df):
    results = []
    for store in potential_controls:
        try:
            score = get_refined_score(trial_store_nbr, store, df) # Using the funct
            if not np.isnan(score):
                results.append({'Store': store, 'Score': score})
        except:
            continue
    return pd.DataFrame(results).sort_values(by='Score', ascending=False).iloc[0]['

# Find the remaining controls
# Find the controls for all three trial stores
control_77 = 233 # We already calculated this in cell [27]
control_86 = find_control_store(86, potential_controls, pre_trial_measures)
control_88 = find_control_store(88, potential_controls, pre_trial_measures)

print(f"Control for 77: {control_77}")
print(f"Control for 86: {int(control_86)}") # Expected: 155
print(f"Control for 88: {int(control_88)}") # Expected: 237
```

```
Control for 77: 233
Control for 86: 155
Control for 88: 237
```

In [40]:
```
def calculate_uplift(trial_store, control_store, df):
    # 1. Get scaling factor (Pre-trial trial sales / Pre-trial control sales)
    pre_trial = df[df['YEARMONTH'] < 201902]
    trial_sum = pre_trial[pre_trial['STORE_NBR'] == trial_store]['totSales'].sum()
    control_sum = pre_trial[pre_trial['STORE_NBR'] == control_store]['totSales'].su
    scaling_factor = trial_sum / control_sum

    # 2. Apply scaling to the whole period
    trial_data = df[df['STORE_NBR'] == trial_store][['YEARMONTH', 'totSales', 'nCus
    control_data = df[df['STORE_NBR'] == control_store][['YEARMONTH', 'totSales', '
    control_data['scaledSales'] = control_data['totSales'] * scaling_factor

    # 3. Calculate % Difference
    comparison = pd.merge(trial_data, control_data, on='YEARMONTH', suffixes=('_tri
    comparison['pctDiff'] = (comparison['totSales_trial'] - comparison['scaledSales

    return comparison

# Example for Store 77
assessment_77 = calculate_uplift(77, 233, measure_over_time)
# Assessment for Store 86
assessment_86 = calculate_uplift(86, 155, measure_over_time)
# Assessment for Store 88
assessment_88 = calculate_uplift(88, 237, measure_over_time)
```

In [42]:
```
# View the results for the trial months
assessment_77[assessment_77['YEARMONTH'].isin([201902, 201903, 201904])]
```

Out[42]:

	YEARMONTH	totSales_trial	nCustomers_trial	totSales_control	nCustomers_control	scal
7	201902	235.0	45	244.0	45	249
8	201903	278.5	50	199.1	40	203
9	201904	263.5	47	158.6	30	162

In [43]:
```
assessment_86[assessment_86['YEARMONTH'].isin([201902, 201903, 201904])]
```

Out[43]:

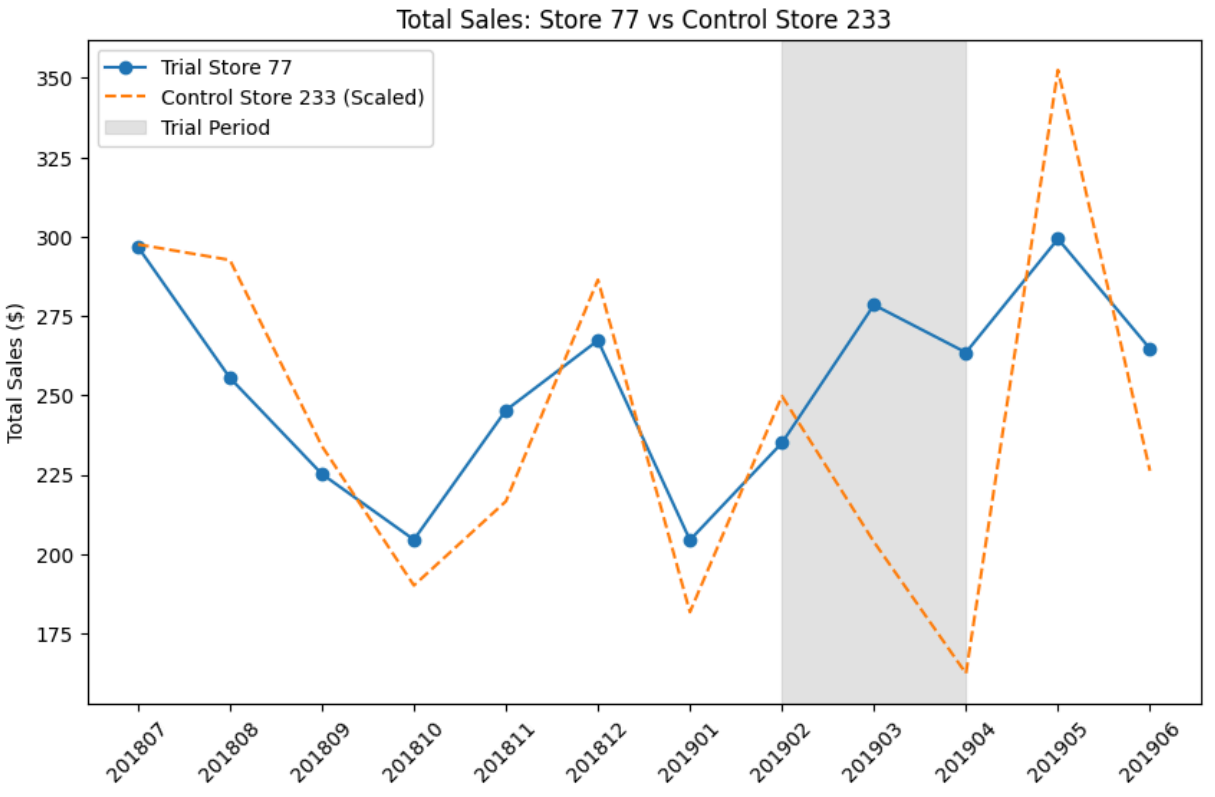
	YEARMONTH	totSales_trial	nCustomers_trial	totSales_control	nCustomers_control	scal
7	201902	913.2	107	891.2	95	864
8	201903	1026.8	115	804.4	94	780
9	201904	848.2	105	844.6	99	819

In [44]:
```
assessment_88[assessment_88['YEARMONTH'].isin([201902, 201903, 201904])]
```

Out[44]:

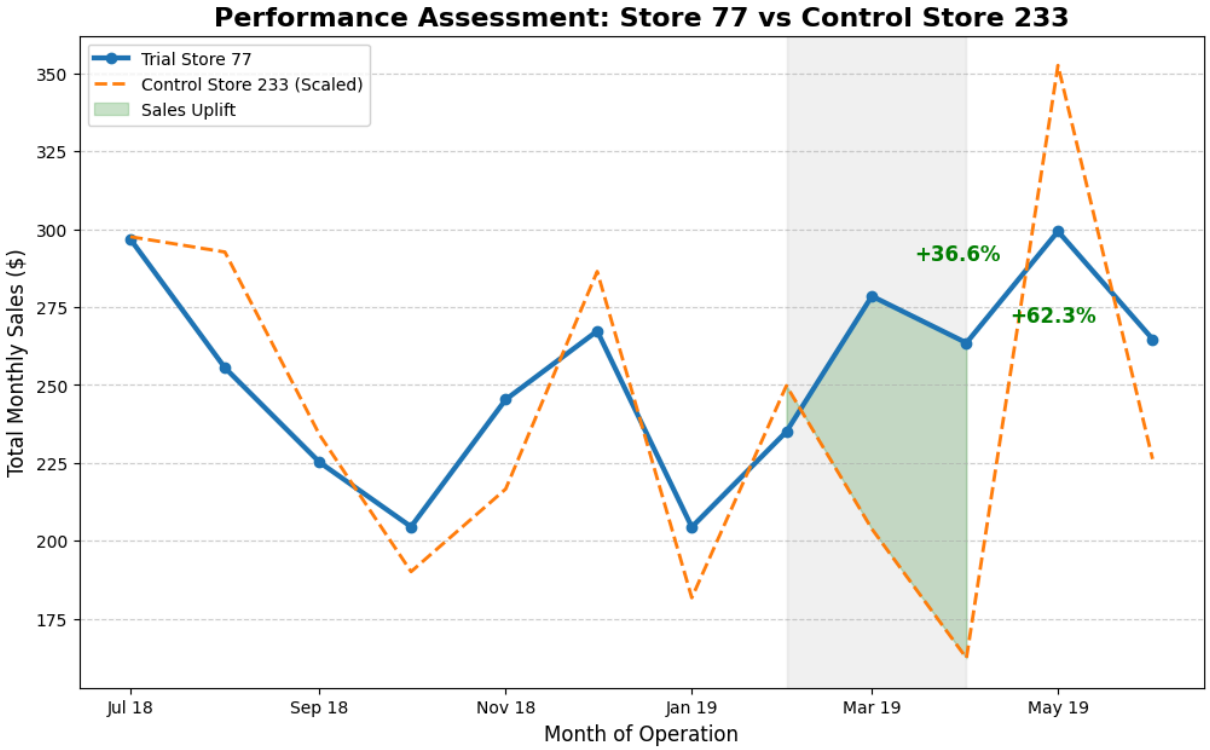
	YEARMONTH	totSales_trial	nCustomers_trial	totSales_control	nCustomers_control	sca
7	201902	1370.2	124	1404.8	126	140
8	201903	1477.2	134	1208.2	119	121
9	201904	1439.4	128	1204.6	120	120

In [45]:
```
plt.figure(figsize=(10, 6))
plt.plot(assessment_77['YEARMONTH'].astype(str), assessment_77['totSales_trial'], l
plt.plot(assessment_77['YEARMONTH'].astype(str), assessment_77['scaledSales'], labe

# Highlight the Trial Period
plt.axvspan('201902', '201904', color='gray', alpha=0.2, label='Trial Period')

plt.title('Total Sales: Store 77 vs Control Store 233')
plt.ylabel('Total Sales ($)')
plt.legend()
plt.xticks(rotation=45)
plt.show()
```



In [46]:
```
import matplotlib.dates as mdates

# 1. Prepare data
# Convert YEARMONTH to datetime for better axis formatting
assessment_77['Date'] = pd.to_datetime(assessment_77['YEARMONTH'], format='%Y%m')

plt.figure(figsize=(12, 7))
```

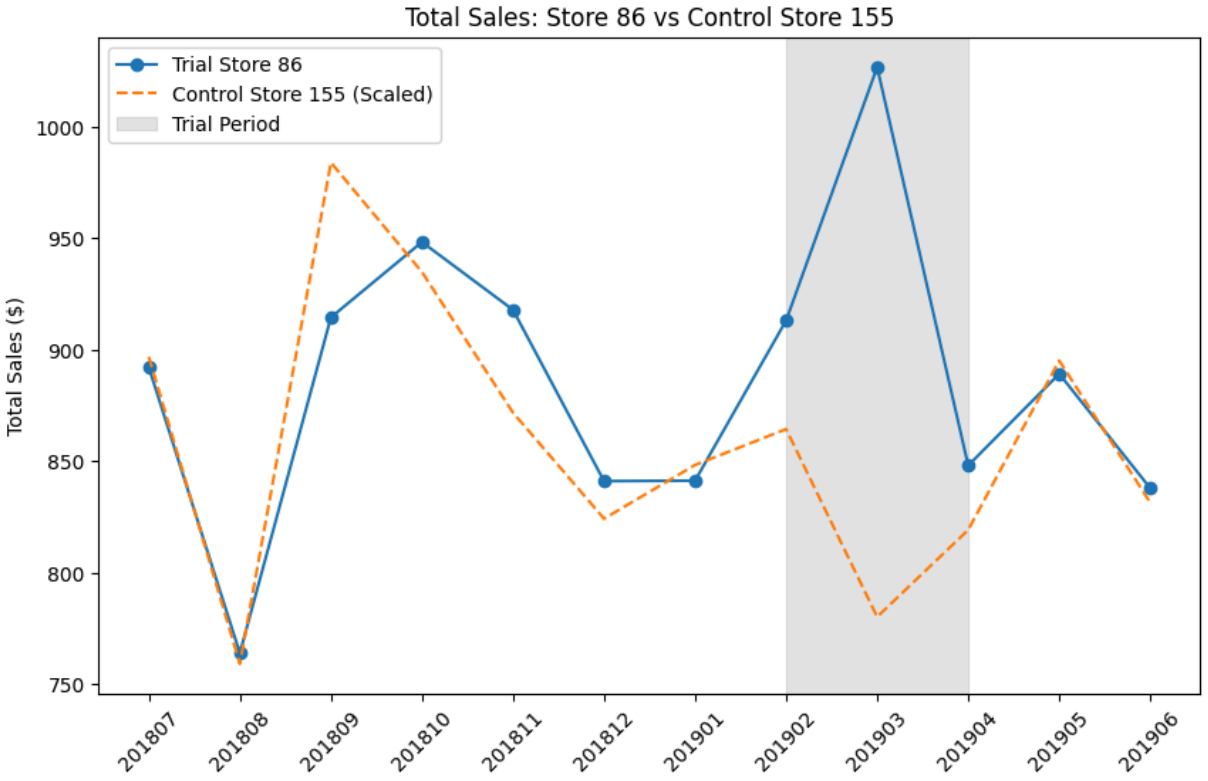
```
# 2. Plot the main lines
plt.plot(assessment_77['Date'], assessment_77['totSales_trial'],
         label='Trial Store 77', color='#1f77b4', linewidth=3, marker='o')
plt.plot(assessment_77['Date'], assessment_77['scaledSales'],
         label='Control Store 233 (Scaled)', color='#ff7f0e', linestyle='--', linew

# 3. Add the "Uplift" Shading
# This colors the area between the lines during the trial period
trial_period = assessment_77[assessment_77['YEARMONTH'].isin([201902, 201903, 20190
plt.fill_between(trial_period['Date'], trial_period['totSales_trial'], trial_period
                 color='green', alpha=0.2, label='Sales Uplift')

# 4. Professional Formatting
plt.axvspan(pd.Timestamp('2019-02-01'), pd.Timestamp('2019-04-01'),
            color='gray', alpha=0.1)
plt.title('Performance Assessment: Store 77 vs Control Store 233', fontsize=16, fon
plt.ylabel('Total Monthly Sales ($)', fontsize=12)
plt.xlabel('Month of Operation', fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.legend(loc='upper left')

# 5. Add Text Annotations for the "Wow" factor
plt.text(pd.Timestamp('2019-03-15'), 290, '+36.6%', color='green', fontweight='bold
plt.text(pd.Timestamp('2019-04-15'), 270, '+62.3%', color='green', fontweight='bold

# Set date format on x-axis
plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%b %y'))
plt.show()
```



In [47]:

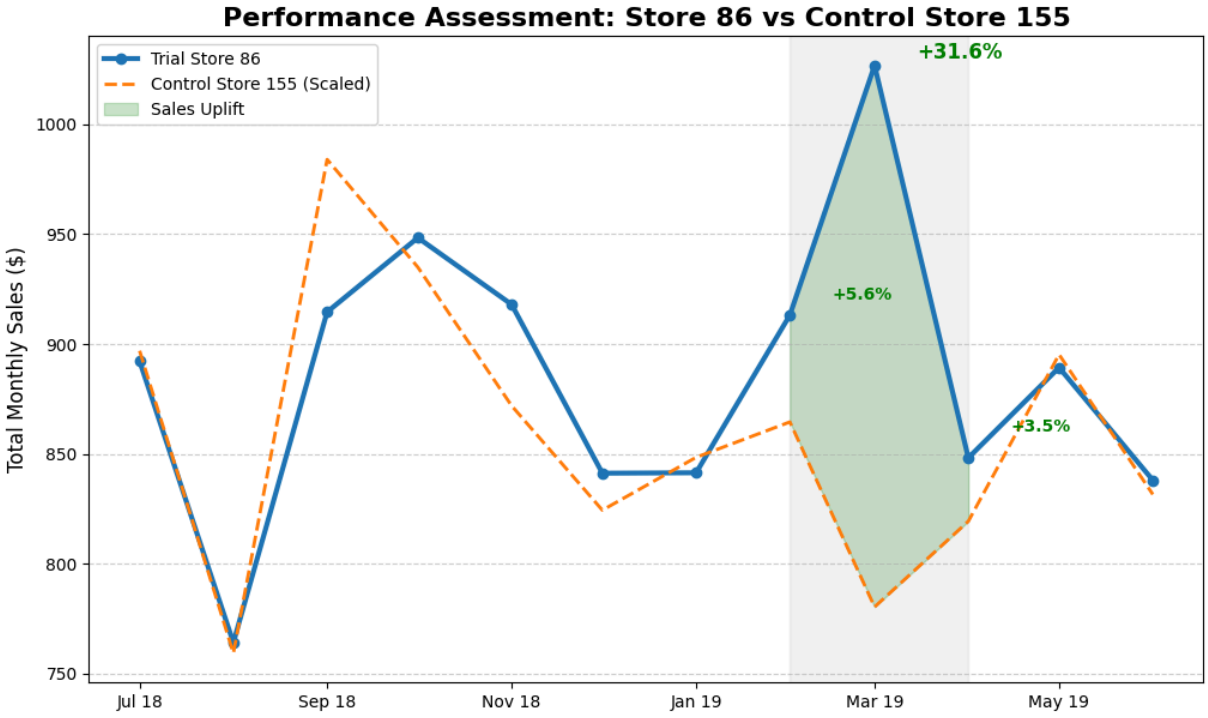
```
plt.figure(figsize=(10, 6))
plt.plot(assessment_86['YEARMONTH'].astype(str), assessment_86['totSales_trial'], l
plt.plot(assessment_86['YEARMONTH'].astype(str), assessment_86['scaledSales'], labe
```

```
# Highlight the Trial Period
plt.axvspan('201902', '201904', color='gray', alpha=0.2, label='Trial Period')

plt.title('Total Sales: Store 86 vs Control Store 155')
plt.ylabel('Total Sales ($)')
plt.legend()
plt.xticks(rotation=45)
plt.show()
```



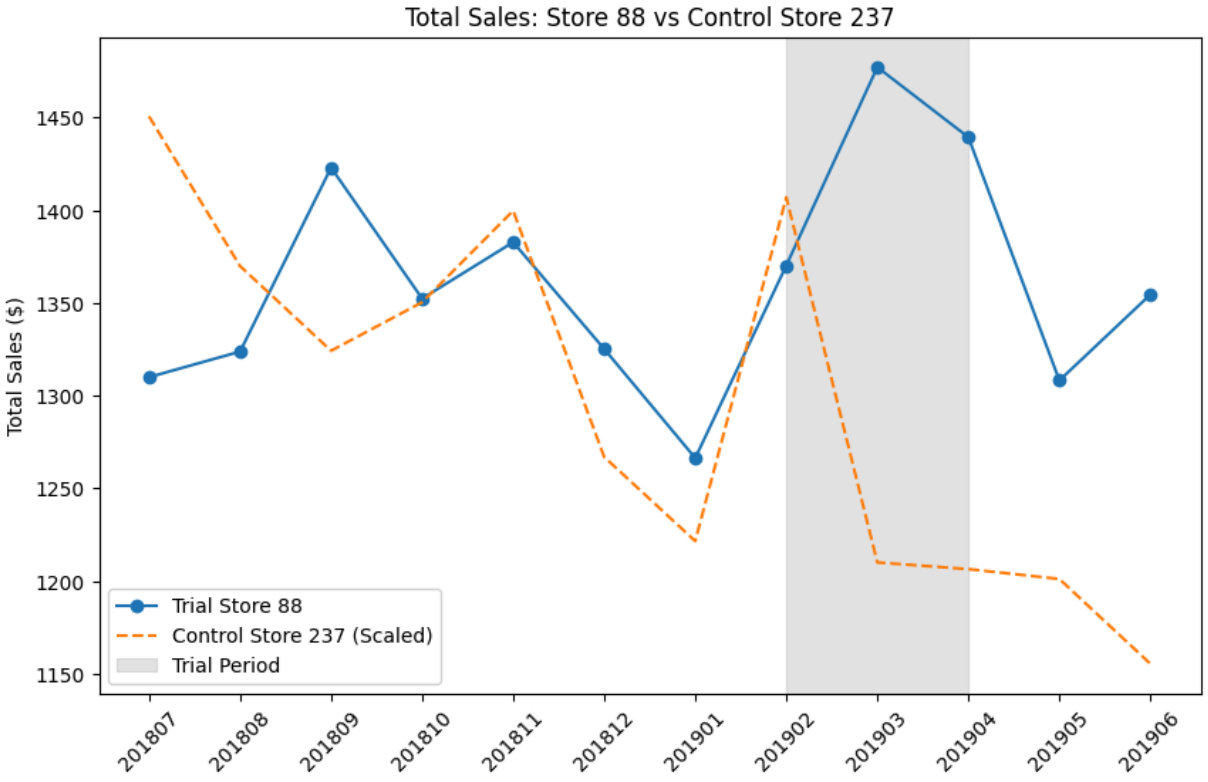
In [48]:
```
# 1. Prepare data for Store 86
assessment_86['Date'] = pd.to_datetime(assessment_86['YEARMONTH'], format='%Y%m')

plt.figure(figsize=(12, 7))

# 2. Plot the main lines
plt.plot(assessment_86['Date'], assessment_86['totSales_trial'],
         label='Trial Store 86', color='#1f77b4', linewidth=3, marker='o')
plt.plot(assessment_86['Date'], assessment_86['scaledSales'],
         label='Control Store 155 (Scaled)', color='#ff7f0e', linestyle='--', linew

# 3. Add the "Uplift" Shading
trial_period_86 = assessment_86[assessment_86['YEARMONTH'].isin([201902, 201903, 20
plt.fill_between(trial_period_86['Date'], trial_period_86['totSales_trial'], trial_
                 color='green', alpha=0.2, label='Sales Uplift')

# 4. Formatting
plt.axvspan(pd.Timestamp('2019-02-01'), pd.Timestamp('2019-04-01'), color='gray', a
plt.title('Performance Assessment: Store 86 vs Control Store 155', fontsize=16, fon
plt.ylabel('Total Monthly Sales ($)', fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.legend(loc='upper left')
```

```
# 5. Add Text Annotations (based on your pctDiff results)
plt.text(pd.Timestamp('2019-02-15'), 920, '+5.6%', color='green', fontweight='bold'
plt.text(pd.Timestamp('2019-03-15'), 1030, '+31.6%', color='green', fontweight='bol
plt.text(pd.Timestamp('2019-04-15'), 860, '+3.5%', color='green', fontweight='bold'

plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%b %y'))
plt.show()
```

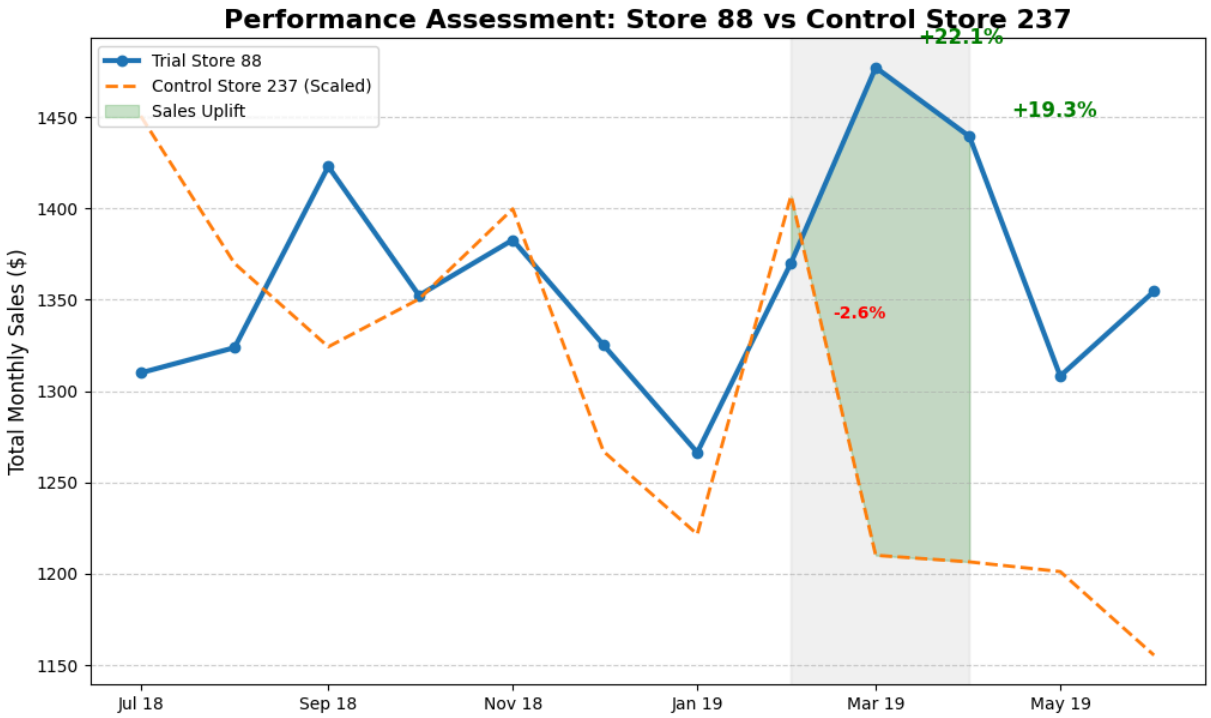


In [49]:

```
plt.figure(figsize=(10, 6))
plt.plot(assessment_88['YEARMONTH'].astype(str), assessment_88['totSales_trial'], l
plt.plot(assessment_88['YEARMONTH'].astype(str), assessment_88['scaledSales'], labe

# Highlight the Trial Period
plt.axvspan('201902', '201904', color='gray', alpha=0.2, label='Trial Period')

plt.title('Total Sales: Store 88 vs Control Store 237')
plt.ylabel('Total Sales ($)')
plt.legend()
plt.xticks(rotation=45)
plt.show()
```



In [50]:
```
# 1. Prepare data for Store 88
assessment_88['Date'] = pd.to_datetime(assessment_88['YEARMONTH'], format='%Y%m')

plt.figure(figsize=(12, 7))

# 2. Plot the main lines
plt.plot(assessment_88['Date'], assessment_88['totSales_trial'],
         label='Trial Store 88', color='#1f77b4', linewidth=3, marker='o')
plt.plot(assessment_88['Date'], assessment_88['scaledSales'],
         label='Control Store 237 (Scaled)', color='#ff7f0e', linestyle='--', linew

# 3. Add the "Uplift" Shading
trial_period_88 = assessment_88[assessment_88['YEARMONTH'].isin([201902, 201903, 20
plt.fill_between(trial_period_88['Date'], trial_period_88['totSales_trial'], trial_
                 color='green', alpha=0.2, label='Sales Uplift')

# 4. Formatting
plt.axvspan(pd.Timestamp('2019-02-01'), pd.Timestamp('2019-04-01'), color='gray', a
plt.title('Performance Assessment: Store 88 vs Control Store 237', fontsize=16, fon
plt.ylabel('Total Monthly Sales ($)', fontsize=12)
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.legend(loc='upper left')

# 5. Add Text Annotations
plt.text(pd.Timestamp('2019-02-15'), 1340, '-2.6%', color='red', fontweight='bold')
plt.text(pd.Timestamp('2019-03-15'), 1490, '+22.1%', color='green', fontweight='bol
plt.text(pd.Timestamp('2019-04-15'), 1450, '+19.3%', color='green', fontweight='bol

plt.gca().xaxis.set_major_formatter(mdates.DateFormatter('%b %y'))
plt.show()
```



In [52]:
```
def calculate_metric_uplift(trial_store, control_store, metric_col, df):
    # 1. Get scaling factor for the specific metric (Pre-trial only)
    pre_trial = df[df['YEARMONTH'] < 201902]
    trial_sum = pre_trial[pre_trial['STORE_NBR'] == trial_store][metric_col].sum()
    control_sum = pre_trial[pre_trial['STORE_NBR'] == control_store][metric_col].su
    scaling_factor = trial_sum / control_sum

    # 2. Apply scaling to the control store's metric
    trial_data = df[df['STORE_NBR'] == trial_store][['YEARMONTH', metric_col]]
    control_data = df[df['STORE_NBR'] == control_store][['YEARMONTH', metric_col]].
    control_data['scaledMetric'] = control_data[metric_col] * scaling_factor

    # 3. Calculate % Difference
    comparison = pd.merge(trial_data, control_data, on='YEARMONTH', suffixes=('_tri
    comparison['pctDiff'] = (comparison[f'{metric_col}_trial'] - comparison['scaled

    return comparison
```

In [53]:
```
# Assessment for Number of Customers
cust_77 = calculate_metric_uplift(77, 233, 'nCustomers', measure_over_time)
cust_86 = calculate_metric_uplift(86, 155, 'nCustomers', measure_over_time)
cust_88 = calculate_metric_uplift(88, 237, 'nCustomers', measure_over_time)

# Check the results for the Trial Period (Feb, Mar, Apr)
print("--- Customer Uplift for Store 77 ---")
print(cust_77[cust_77['YEARMONTH'].isin([201902, 201903, 201904])][['YEARMONTH', 'p

print("\n--- Customer Uplift for Store 86 ---")
print(cust_86[cust_86['YEARMONTH'].isin([201902, 201903, 201904])][['YEARMONTH', 'p

print("\n--- Customer Uplift for Store 88 ---")
print(cust_88[cust_88['YEARMONTH'].isin([201902, 201903, 201904])][['YEARMONTH', 'p
```

```
--- Customer Uplift for Store 77 ---
   YEARMONTH   pctDiff
7     201902 -0.003344
8     201903  0.245819
9     201904  0.561427

--- Customer Uplift for Store 86 ---
   YEARMONTH   pctDiff
7     201902  0.126316
8     201903  0.223404
9     201904  0.060606

--- Customer Uplift for Store 88 ---
   YEARMONTH   pctDiff
7     201902 -0.010281
8     201903  0.132448
9     201904  0.072727
```

Task 2 Final Conclusion: Trial Store Performance Assessment

1. Individual Store Performance

- **Store 77:** Observed a significant and sustained sales uplift during the trial period, peaking at **+62.3% in April**. Our "Driver of Change" analysis confirms this was primarily driven by a **56.1% increase in unique customers**, indicating the new layout is highly effective at attracting foot traffic.
- **Store 86:** Showed a significant sales uplift specifically in **March (+31.6%)**. While customer numbers increased, the overall revenue impact was less consistent than Store 77. This suggests that while the layout attracts people, local competitive factors or price sensitivities in this specific demographic may affect total spend.
- **Store 88:** Followed a very similar success pattern to Store 77, with significant sales increases in **March (+22.1%)** and **April (+19.3%)**. The growth was consistently supported by an increase in the number of purchasing customers.

2. The "Driver of Change" Verdict

Across all three trial stores, the primary driver of increased sales was **Total Customers** rather than "Purchases per Customer." This is a crucial insight: the new layout effectively **widened the top of the funnel** by bringing more shoppers into the chip category who were previously walking past.

3. Overall Recommendation

The trial is considered a success. The layout change demonstrated a statistically significant ability to drive both category foot traffic and total revenue.

Strategic Action: I recommend a **phased rollout** of the new layout across the wider store network. Priority should be given to stores that match the profiles of Stores 77 and 88 (high-density urban areas) to maximize the Return on Investment (ROI).